🏗️ Projet Cockpit V2 — Spécification Technique Complète

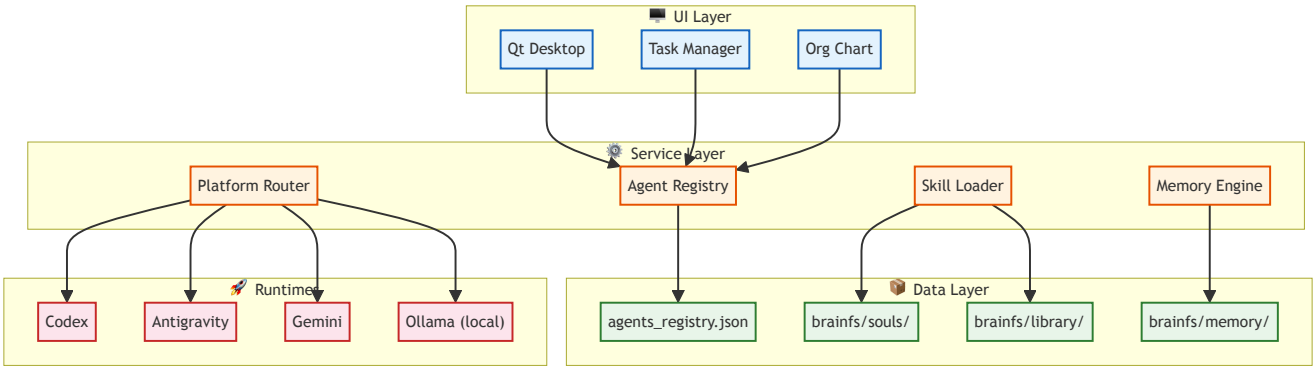
> **Document de référence pour Clems (Orchestrator L0)**
> **Auteur :** Léo (Agent L1 — Recherche & Architecture)
> **Date :** 2026-02-13
> **Statut :** Validé par Olivier — Prêt pour implémentation

1. Contexte & Justification

1.1 Les 5 Murs de la V1

#	Mur	Preuve dans le code	Impact
1	**Agents hardcodés**	`auto_mode.py:296` — `_agent_platform()` = if/else statique	🔴 Impossible d'ajouter un agent sans modifier Python
2	**Pas de mémoire cross-projet**	`brainfs.py:63` — `load_profile()` lit UN profil par projet	🔴 Amnésie entre projets
3	**Skills décoratives**	`brainfs.py:19` — `load_skills()` → jamais injecté dans le prompt	🟡 Les compétences ne changent rien
4	**Dispatch fragile**	`auto_mode.py:458` — 1 requête à la fois, synchrone	🔴 Requête perdue si crash
5	**Single Runtime**	`auto_mode.py:296` — 5/7 agents sur Codex	🔴 80% bloqué si Codex down

1.2 Architecture Cible



2. Les 5 Plugins

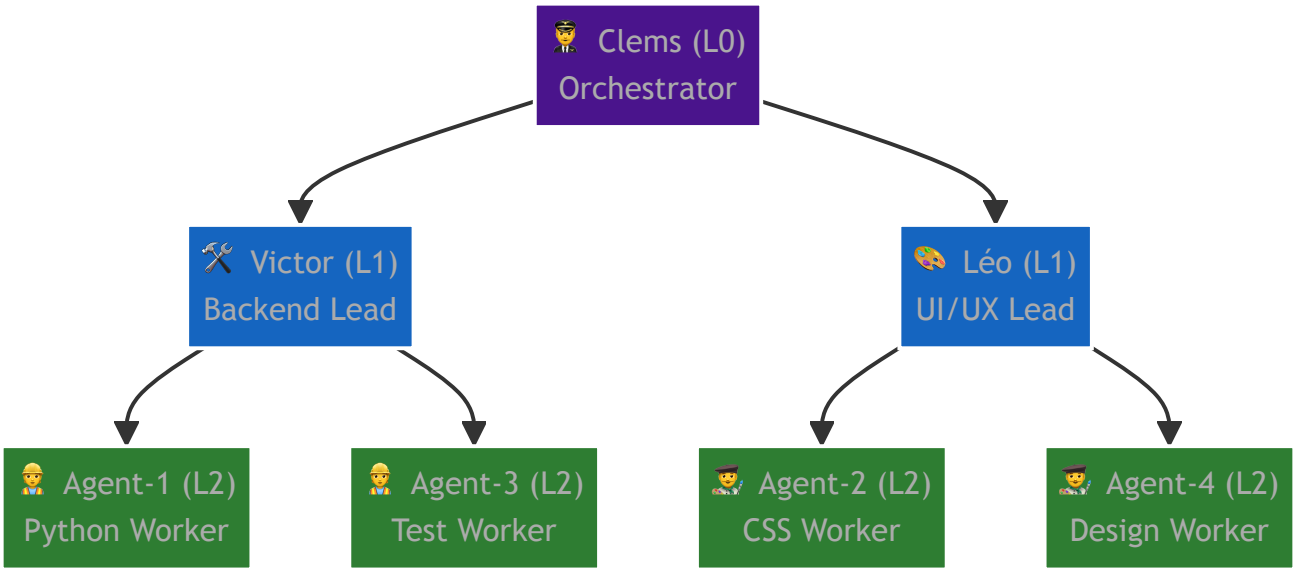
PHASE 1 Plugin 1 : Agent Registry & Hiérarchie

Pourquoi : On ne peut pas ajouter un agent sans modifier le code Python. `_agent_platform()` est un if/else hardcodé.

Solution : Un fichier `brainfs/agents_registry.json` — source de vérité unique pour tous les agents.

```
{
  "agents": {
    "clems":  { "level": 0, "role": "Orchestrator",    "platform": "antigravity", "m
    "victor": { "level": 1, "role": "Backend Lead",     "platform": "codex",       "
    "leo":    { "level": 1, "role": "UI/UX Lead",       "platform": "antigravity", "
    "agent-1":{ "level": 2, "role": "Python Worker",    "platform": "codex",       "
  }
}
```

Hiérarchie résultante :



Fichier	Action	Effort
`brainfs/agents_registry.json`	CRÉER	1h
`app/services/brainfs.py`	MODIFIER — ajouter `load_agent_registry()`	2h
`app/services/auto_mode.py`	MODIFIER — `_agent_platform()` → lookup registry	3h
`app/data/model.py`	MODIFIER — ajouter `level`, `parent_id`	1h
`app/ui/agents_grid.py`	MODIFIER — grouper par niveau	4h

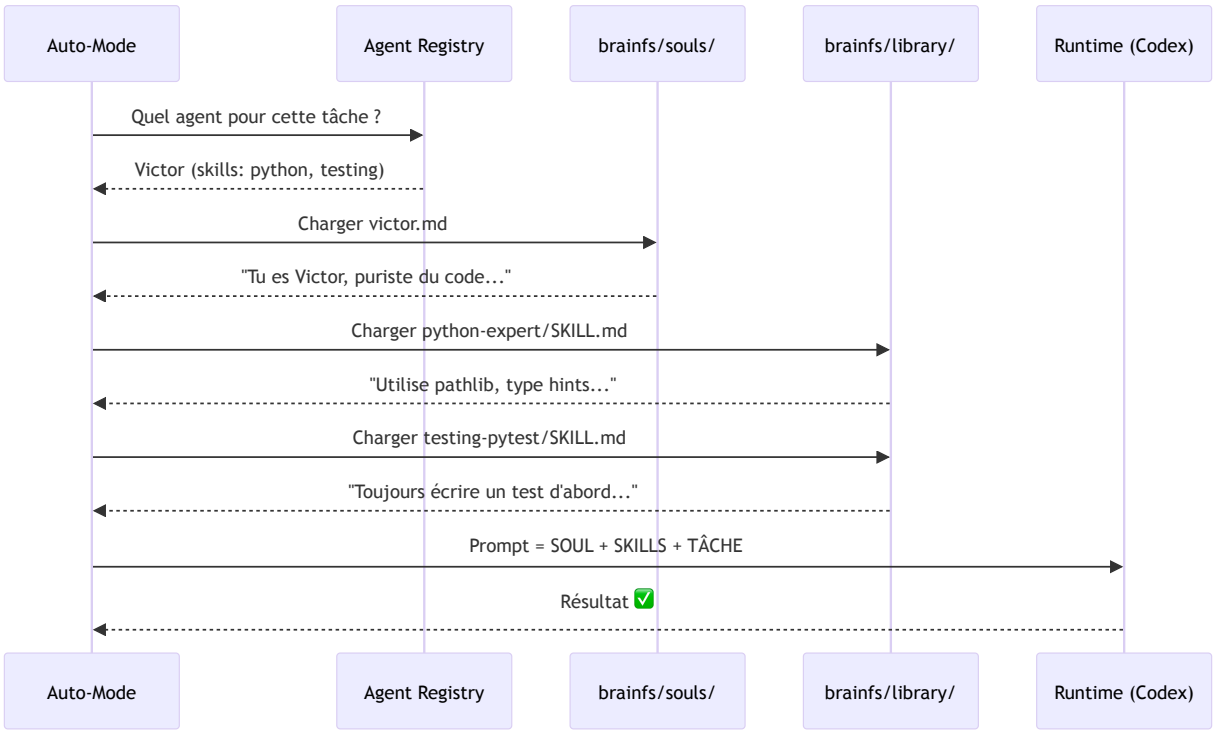
PHASE 1 Plugin 2 : SOUL.md & Skills Dynamiques

Pourquoi : Les agents n'ont pas de personnalité. `skills.json` n'est jamais injecté dans les prompts.

Solution :

- **SOUL.md** = fiche d'identité de chaque agent (ton, règles, mémoire)
- **Skill Library** = `brainfs/library/` avec des dossiers par compétence contenant `SKILL.md`

Flux d'injection :



Fichier	Action	Effort
`brainfs/souls/clems.md`	CRÉER	1h
`brainfs/souls/victor.md`	CRÉER	1h
`brainfs/souls/leo.md`	CRÉER	1h
`brainfs/library/python-expert/SKILL.md`	CRÉER	2h
`brainfs/library/qt-styling/SKILL.md`	CRÉER	2h
`app/services/brainfs.py`	MODIFIER — `load_soul()`, `load_skills_for_agent()`	3h
`app/services/auto_mode.py`	MODIFIER — `format_prompt()` injecte SOUL + Skills	3h

PHASE 1 Plugin 3 : Task Manager Simplifié

Pourquoi : L'auto-mode est une boîte noire. Aucune visibilité sur qui fait quoi.

Solution : Un tableau (type "panneau d'aéroport") dans l'UI.

Agent	Statut	Mission	Depuis
👨‍✈️ Clems	🟢 En Mission	Orchestration projet V2	2 min
🛠️ Victor	🟡 En Attente	Attend review Olivier	15 min
🎨 Léo	⚪ Au Repos	—	—
👷 Agent-1	🟢 En Mission	Fix CI pipeline	8 min

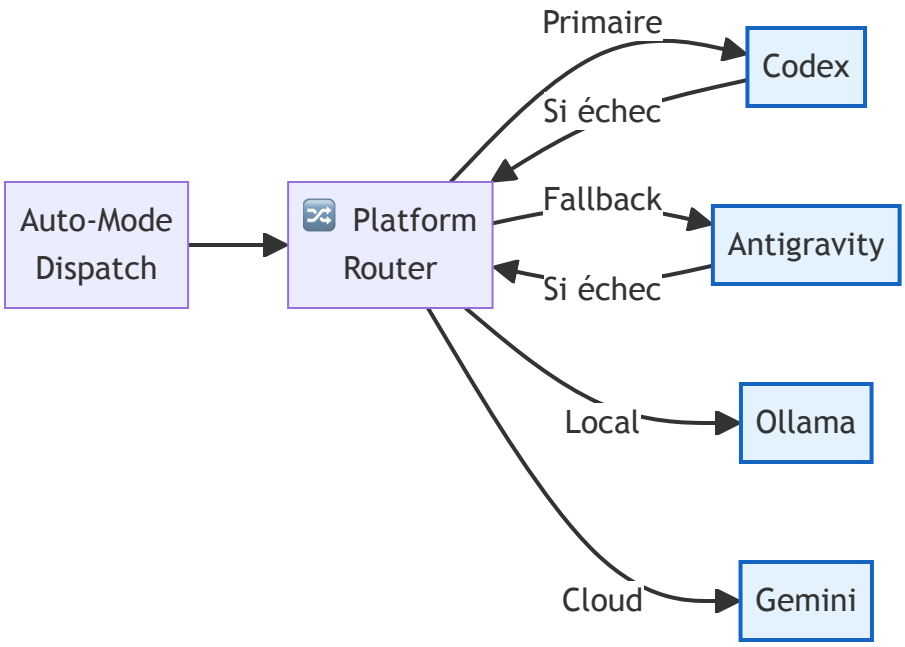
Pas de tokens. Pas de coûts. (Phase 3 si nécessaire.)

Fichier	Action	Effort
`app/ui/task_manager.py`	CRÉER — widget `QTableWidget`	4h
`app/ui/sidebar.py`	MODIFIER — ajouter entrée "Mission Control"	2h
`app/main.py`	MODIFIER — intégrer dans le stack central	2h

PHASE 2 Plugin 4 : Platform Router (Multi-Runtime)

Pourquoi : 5/7 agents dépendent de Codex. Single point of failure.

Solution : Un routeur qui distribue les tâches au bon runtime avec fallback automatique.



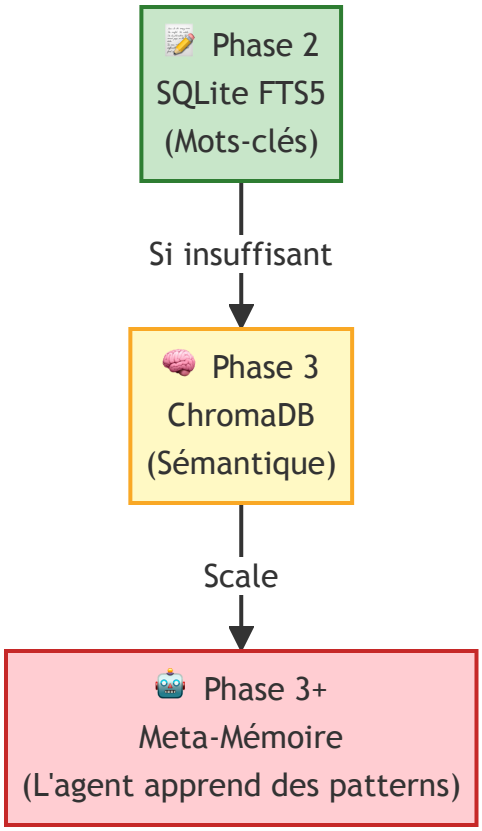
Fichier	Action	Effort
`app/services/platform_router.py`	CRÉER — interface `RuntimeProvider` + Router	5h
`app/services/providers/codex.py`	CRÉER — wrapper existant	3h
`app/services/providers/antigravity.py`	CRÉER — wrapper existant	2h
`app/services/providers/ollama.py`	CRÉER — nouveau	4h
`app/services/auto_mode.py`	MODIFIER — utiliser Router au lieu de `_agent_platform()`	4h

PHASE 2 Plugin 5 : Mémoire Progressive

Pourquoi : Les agents oublient tout entre les sessions. Pas de recherche cross-projet.

Étape A (Phase 2) — SQLite FTS5 : Recherche par mots-clés (0 dépendance externe).

Étape B (Phase 3) — ChromaDB : Recherche sémantique (comprend le sens, pas juste les mots).



Fichier	Action	Effort
`brainfs/memory/`	CRÉER (dossier)	—
`app/services/memory_engine.py`	CRÉER — `index()`, `search()`, `learn_pattern()`	4h
`app/data/store.py`	MODIFIER — hook indexation auto	2h
`app/services/auto_mode.py`	MODIFIER — indexation post-dispatch	2h

3. Prérequis Hardware

3.1 Configuration actuelle (Mac d'Olivier)

Composant	Minimum	Recommandé	Rôle
CPU	Apple M1	Apple M2 Pro / M3	Qt + SQLite FTS
RAM	8 GB	16 GB	Qt + 3 agents Codex simultanés
Stockage	10 GB libres	50 GB SSD	Projets + brainfs + memory
Réseau	Connexion stable	Fibre	Appels API Codex/AG/Gemini

> ✅ **Un MacBook Pro M1/M2 16 GB suffit pour tout sauf le Vector Store lourd.**

3.2 Si Ollama (LLM local)

Modèle	Taille	RAM requise	Qualité	Vitesse
`llama3.2:3b`	2 GB	4 GB	⭐⭐	⚡⚡⚡
`llama3.2:7b`	4 GB	8 GB	⭐⭐⭐	⚡⚡
`codellama:13b`	7 GB	16 GB	⭐⭐⭐⭐	⚡
`deepseek-coder:6.7b`	4 GB	8 GB	⭐⭐⭐⭐ (code)	⚡⚡

3.3 Serveur Privé — Est-ce nécessaire ?

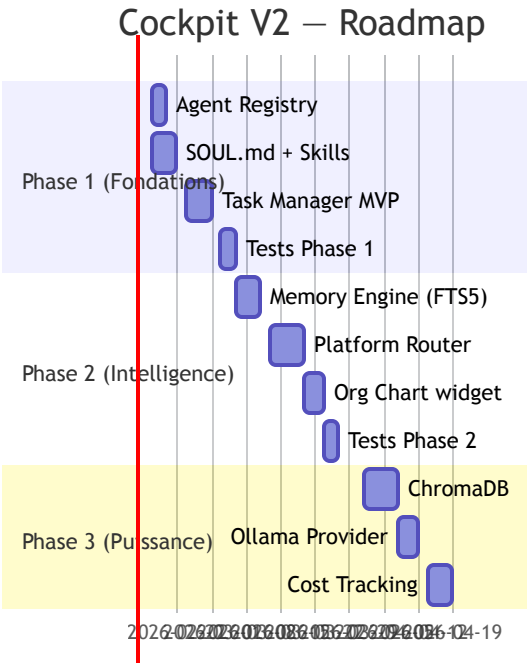
> **NON, pas pour l'instant.** Le Mac d'Olivier suffit pour tout sauf un modèle 70B+.

Si on décide d'en prendre un (plus tard) :

Type	Fournisseur	Spécifications	Prix/mois	Cas d'usage
VPS simple	Hetzner / OVH	4 vCPU, 16 GB RAM, 100 GB SSD	~20-30 €	Cron jobs, mémoire centralisée
GPU Cloud	Lambda Labs / Vast.ai	1x RTX 4090, 24 GB VRAM	~100-300 €	LLM local gros (70B)
🍎 **Mac Mini M4**	Apple	M4, 24 GB unifiée, 512 GB	**~800 € (achat unique)**	LLM local, 0 coût récurrent

> 💡 **Recommandation :** Si tu veux un serveur, achète un **Mac Mini M4 24 GB**. Achat unique, silencieux, fait tourner tout 24/7.

4. Roadmap Résumée



Phase	Durée	Effort	Résultat
PHASE 1	2 semaines	~30h	Agents identifiés, Skills, Task Manager MVP

PHASE 2	4 semaines	~31h	Mémoire FTS, Platform Router, Org Chart
PHASE 3	8 semaines	~34h	Vector Store, LLM local, Coûts
	~3 mois	**~95h**	**Cockpit V2 complet**

5. Risques & Mitigations

#	Risque	Probabilité	Impact	Mitigation
1	Permissions macOS	Élevée	🔴	Stocker dans le dossier projet
2	ChromaDB trop lourd	Faible	🟡	SQLite FTS5 comme plan B permanent
3	Ollama = trop de RAM	Moyenne	🟡	Modèles < 7B, décharger après usage
4	Prompt trop long	Moyenne	🔴	Budget token par section
5	Migration auto_mode casse le dispatch	Élevée	🔴	Feature flags + tests avant merge
6	Codex rate limiting	Moyenne	🟡	Platform Router + fallback + queue

> **Ce document est la feuille de route complète pour Cockpit V2.**
> Clems peut l'utiliser pour distribuer les tâches, phase par phase.
> Aucun codage ne commence avant validation par Olivier.